CI CD Jenkins Cloud DevOps Hub Notes

11 October 2026 13:48

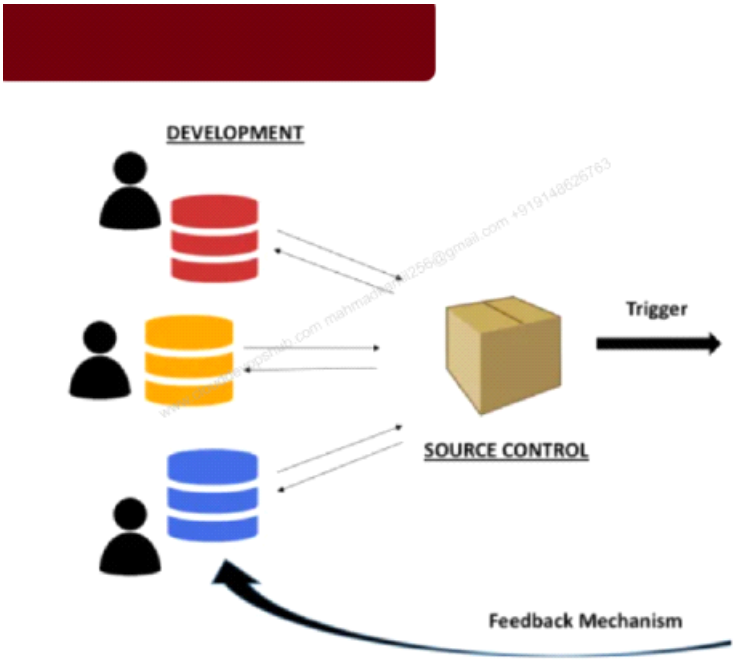


What all things we will cover in this session

- Post using GIT in Dev environment?
- Code managed and then create a build and make this process automatic via Continuous integration.
- What is Continuous integration.
- Traditional VS Continuous integration
- Importance of Continuous integration
- Popular Continuous integration tools in market
- Why Jenkins Popularity as per google ?
- What is Jenkins?
- Why/how to use Jenkins?
- Jenkins Work flow
- What all things Jenkins can do?
- Jenkins Installation
  - Jenkins application installation
  - Jenkins run via war file
  - Jenkins application installation on docker container
  - Jenkins installation on Ubuntu Machine
- Jenkins Job creation and Job Types
- Different types of Jenkins Jobs
- Practical's Job 1- Pulling Java project from git and compile program via Jenkins
- Job 2 – Compile and Building Java program / project from git
- Job 3- Simple Java program compile java program locally if you have installed Jenkins locally
- What is Jenkins Pipeline?
- Job 4- Create Jenkins pipeline CI-example
- Schedule a build Periodically
- Jenkins Useful Top Plugins out of 1500+

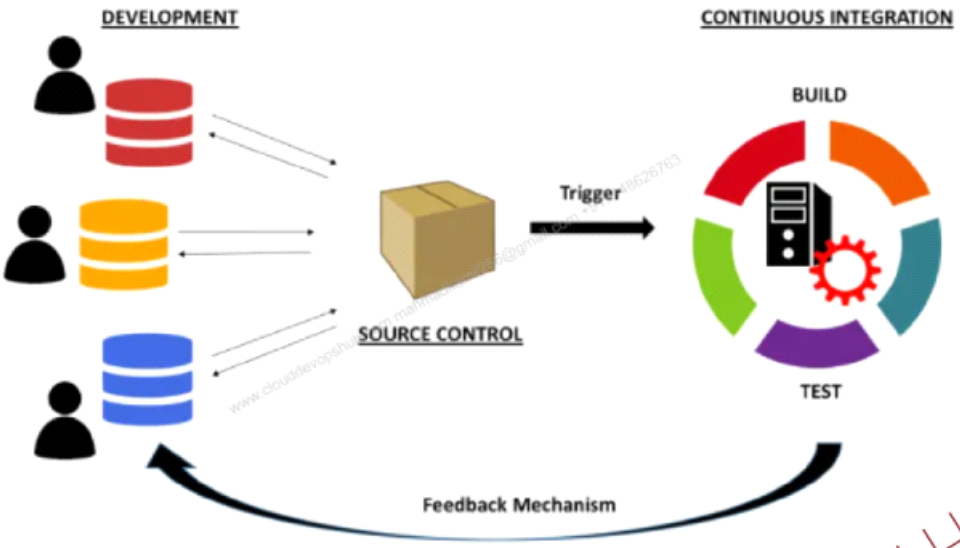
LIVE zoom





Post Configuring GIT in dev environment ?

Continous Integration's



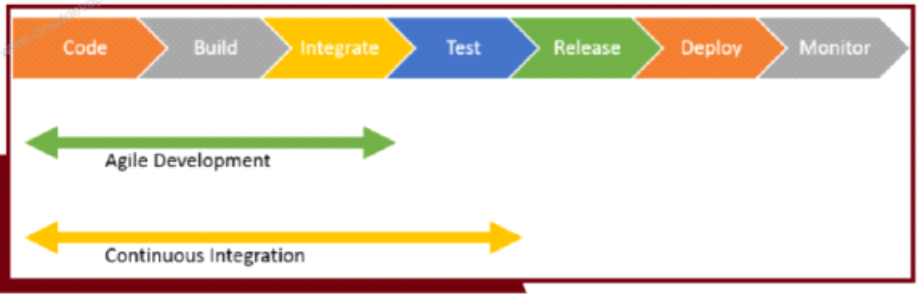
What is Continuous Integration?

Continuous Integration (CI) is a development practice where developers **integrate** code into a shared repository frequently, preferably several times a day.

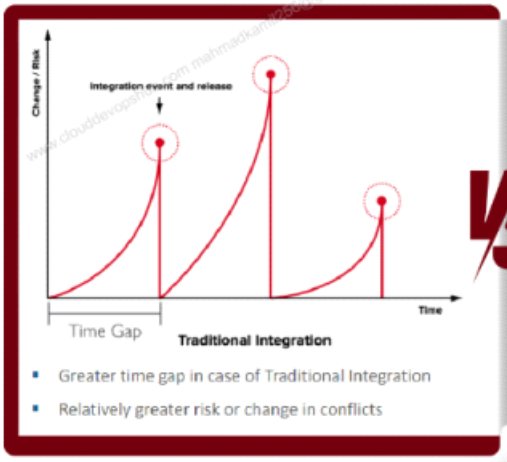
Each **integration** can then be verified by an automated build and automated tests {Unit tests}.

Basically Continuous Integration is the practice of running your builds and tests automatically every time when someone pushes new code into the source repository (Github).

This has the tremendous advantage of always knowing if all tests work and getting fast feedback.

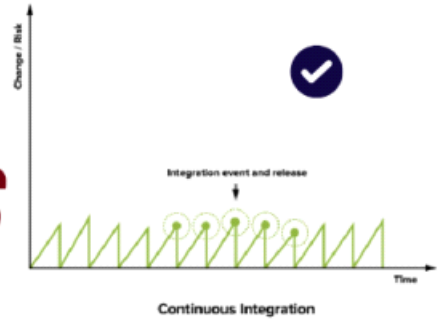


Traditional



- Greater time gap in case of Traditional Integration
- Relatively greater risk or change in conflicts

Continuous Integration



- Integration time gap is relatively much lesser
- Relatively lesser risk or change in conflicts

Importance Of Continuous Integration

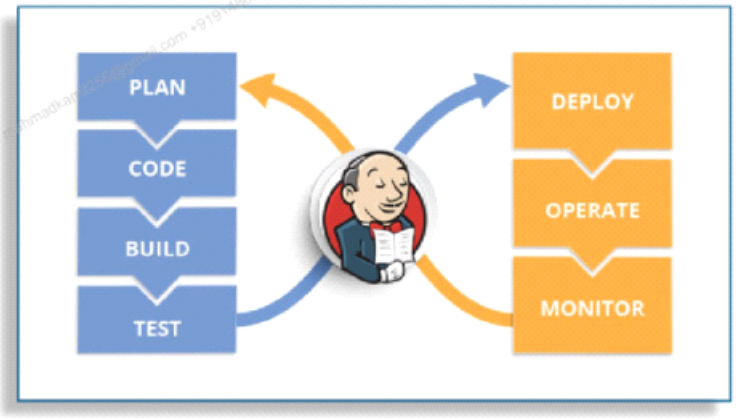


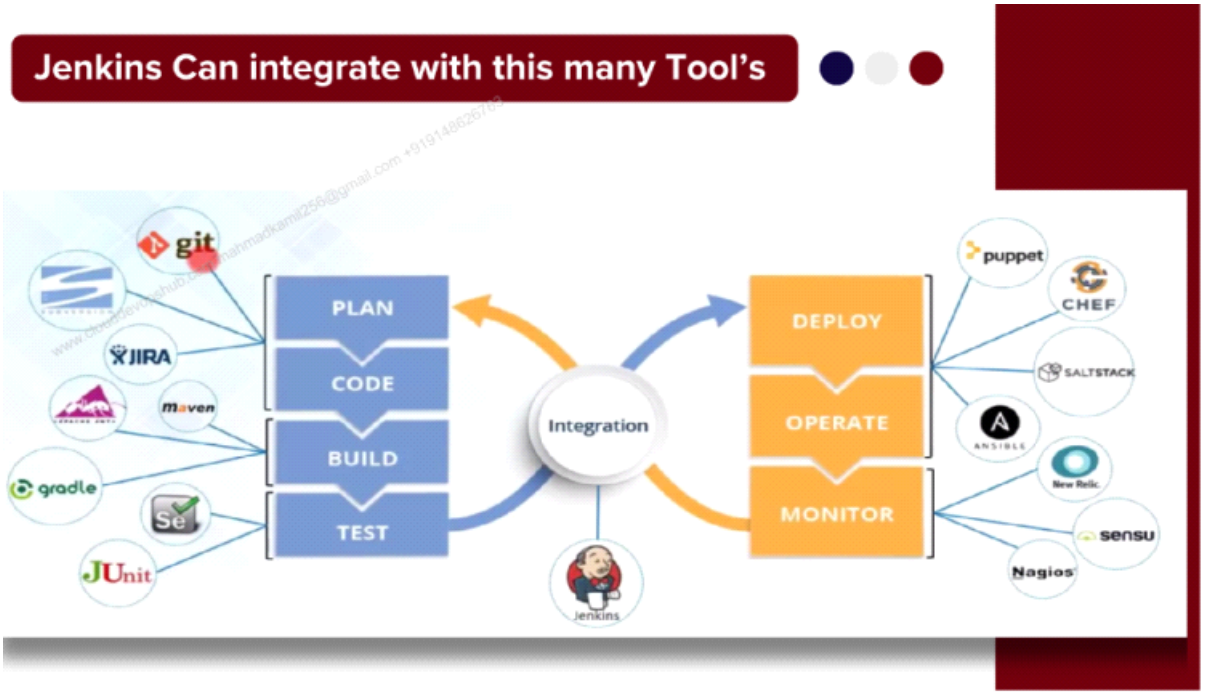
Popular Continuous Integration Tools



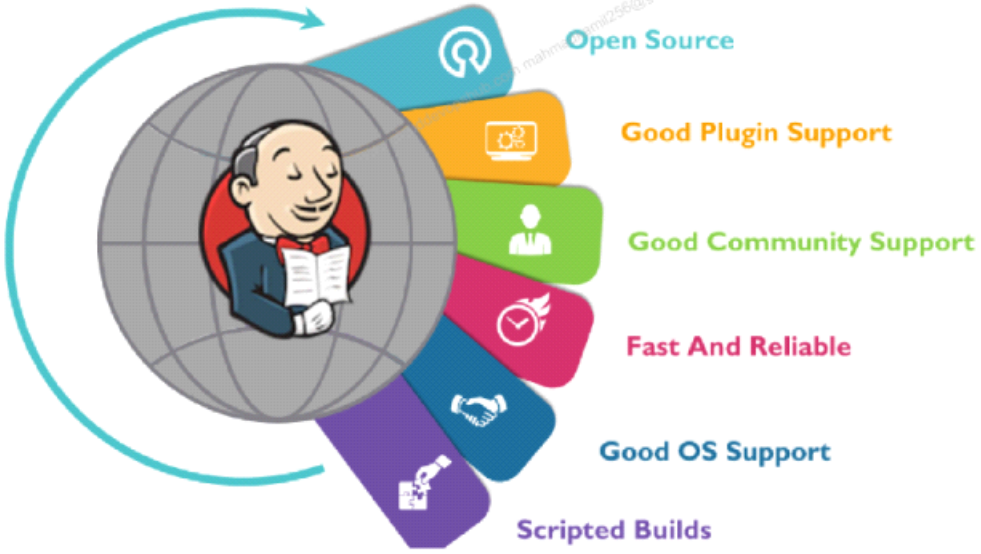
What Is Jenkins?

"A Continuous Integration server which manages and control processes such as plan, code, build, test, dep operate and monitor in DevOps Environment."

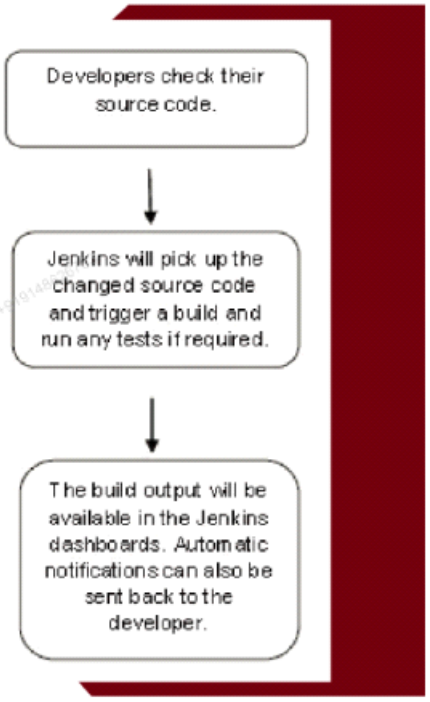




Why Jenkins Is So Popular?



Jenkins is a software that allows continuous integration
Simple work flow of Jenkins :



What all things Jenkins can manage??

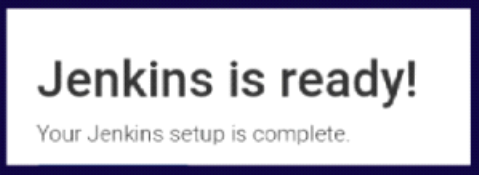
- Source control management then > Builds + Unit Testing
- Manual works /Activity
- Schedule work - Corn Jobs in unix
- Notifications/ mails to stockholder
- Automations (running scripts)
- Dependent work via pipelines



Jenkins Installation

Approach 1 as a End User

Jenkins Application installation locally means download and install via
https://www.jenkins.io/download/
localhost:8080



Approach 2 as a Developer / DevOps

Run Jenkins from war file (without installation) Run the war file

https://updates.jenkins-ci.org/download/war/

Download JAVA
https://docs.oracle.com/javase/8/docs/technotes/guides/install/windows_system_requirements.html#BABHGJJF

https://www.java.com/download/ie_manual.jsp
Set path as per this document
https://www.java.com/en/download/help/path.xml

Create Dev folder in C drive and paste war file in Dev folder open CMD and run below command Jenkins 2.319.2




```
java -jar jenkins.war
java -jar jenkins.war --httpPort=8082        --- if you want to run on any specific path
```

Running from: C:\Dev\jenkins.war

Jenkins installation…

Approach 3

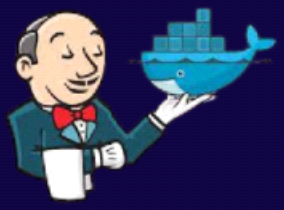
Jenkins Application installation on docker container...

Step 1:- Pull docker image and run the Jenkins form below command on port 8080

```
docker run -it -d -p 8080:8080 --name jenkins jenkins/jenkins:latest
```

Step 2 :- Check Jenkins container running status.
Docker ps

Step 3 :- Login to Jenkins container and take the admin password and create user.



```
docker exec –it <container ID>  bin/bash

cat /var/jenkins_home/secrets/initialAdminPassword
```

Step 4 :- Login and application is ready for Continuous integration.

Approach 4 :- Install Jenkins On Ubuntu (this is production / real time approach)

MOST POPULAR

STEP'S

This installation is specific to systems operating on Ubuntu. Follow the below steps:
https://www.jenkins.io/doc/book/installing/linux/

Step 1: Install Java $ sudo apt update verify java -version
$ sudo apt install openjdk-17-jre-headless
sudo apt install openjdk-21-jdk -y with JDK

Step 2: Add Jenkins Repository i.e key
```
curl -fsSL https://pkg.jenkins.io/debian/jenkins.io-2023.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null
```
Step 3: Add Jenkins repo to the system i.e source list
```
$  echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null
```
Step 4: Install Jenkins
$ sudo apt update
$ sudo apt install jenkins

Step 5: Verify installation
$ systemctl status jenkins

Step 6: Once Jenkins is up and running, access it from the
link: http://localhost:8080 or the IP of virtual machine like xx.xx.xx.xx:8080

Get the initial admin pwd from command cat /var/lib/jenkins/secrets/initialAdminPassword

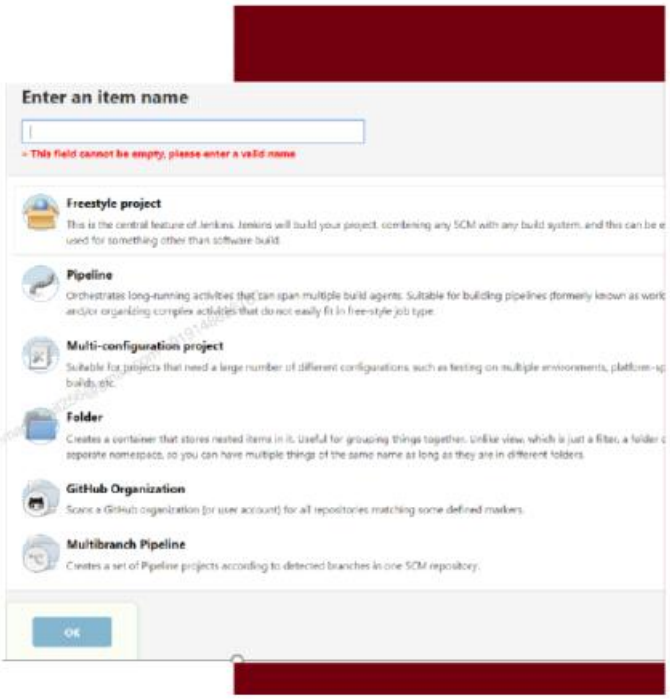
Start, Stop & Restart Jenkins

$ sudo service jenkins restart
$ sudo service jenkins stop
$ sudo service jenkins start



Jenkins Job Creation page & Types of Jobs





Different Types of Jenkins Jobs

Freestyle Job
Jenkins provides the option of choosing from different types of jobs to build your project.
Freestyle build jobs are general-purpose build jobs, which provides maximum flexibility. It can be used for any type of project.

Pipeline
This project runs the entire software development workflow as code. Instead of creating several jobs for each stage of software development, you can now run the entire workflow as one code.

Multiconfiguration
The multiconfiguration project allows you to run the same build job on different environments. It is used for testing an application in different environments.

Folder
This project allows users to create folders to organize and categorize similar jobs in one folder or sub folder.

GitHub Organisation
This project scans your entire GitHub organization and creates Pipeline jobs for each repository containing a Jenkinsfile

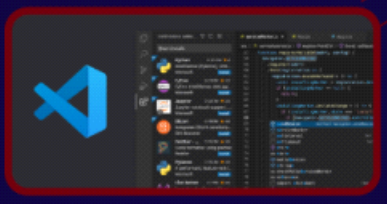
Multibranch Pipeline
This project type lets you implement different Jenkinsfiles for different branches of the same project.

Steps Run Simple Java program manually via Visual Studio Code

Step 1 Create helloworld.java file and paste the step 2 code or Open Visual stdio code

Step 2 Code

```
class Simple{
   public static void main(String args[]){
    System.out.println("Hello Students from Batch-34");
   }
}
```



Open Terminal Ctrl + Shift + ~

Step 3 compile java program by command Javac helloworld.java



Step 4 verify is class name compiled file created or not if yes run a java program by Java Simple command

Step 5 Check result/output

Output will be "Hello Java"

Note : the compiler will generate the .class file with the exact same name given to the Java class in the program

Interview Question :- Building vs. Compiling (Java program)



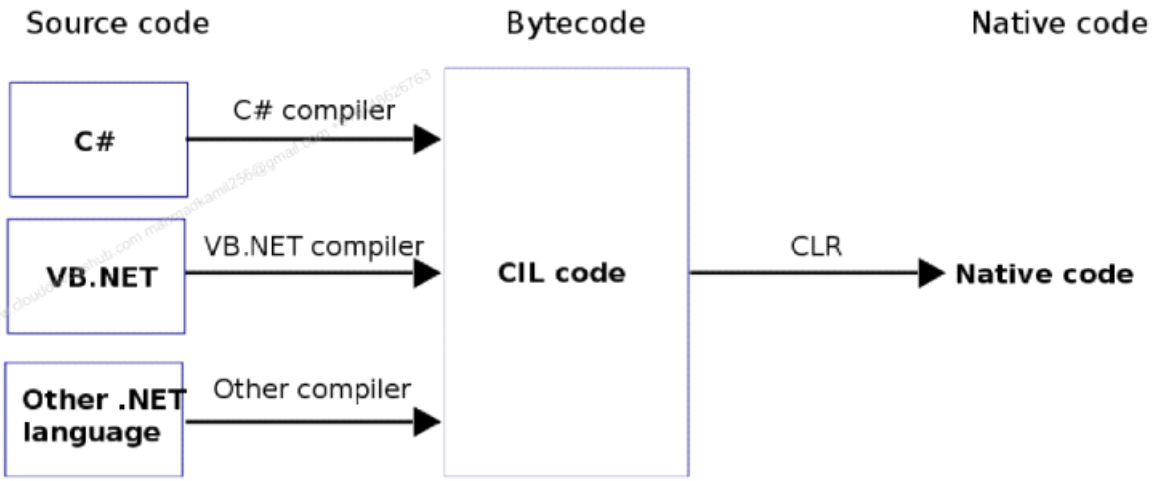
Small program we compile it
Big program/project we Compile + Build it.

The "Build" is a process that covers all the steps required to create a "deliverable" of your software.
In the Java world, this typically includes:

- Generating sources (sometimes).
- Compiling sources.
- Compiling test sources.
- Executing tests (unit tests, integration tests, etc).
- Packaging (into jar, war, ejb-jar, ear).
- Running health checks (static analyzers like Checkstyle, Findbugs, PMD, test coverage, etc).
- Generating reports.



All other Languages

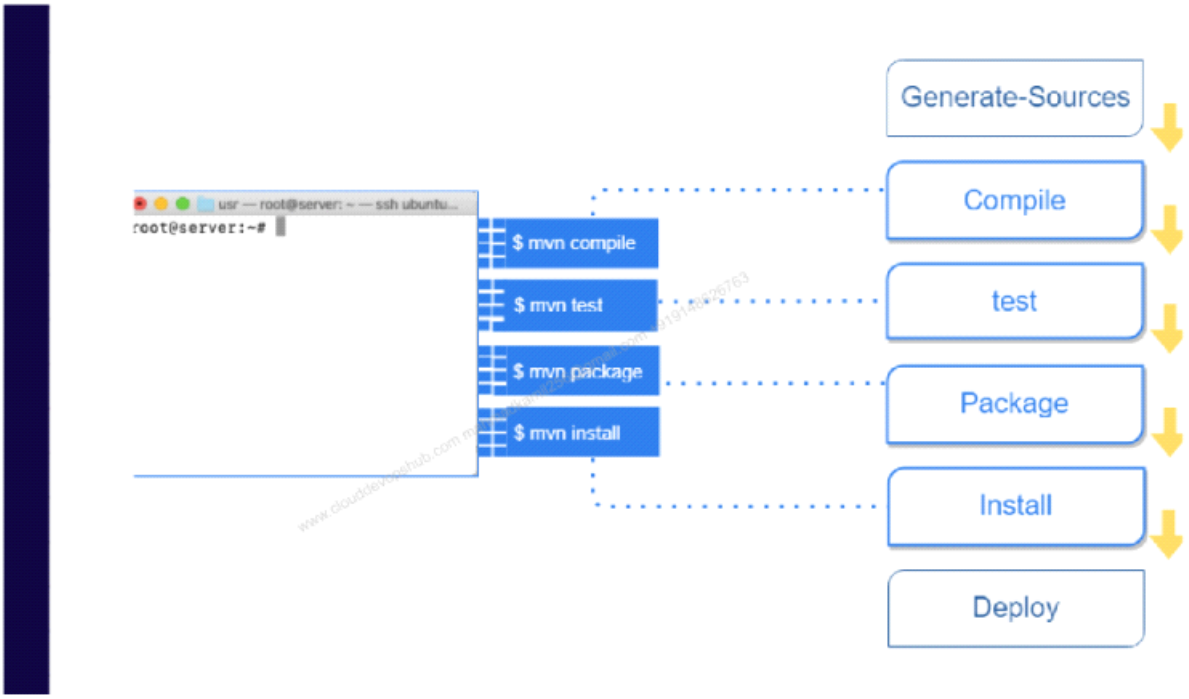


.NET uses two compilers, Roslyn, to compile C# or VB code into CIL (common intermediate language), and RyuJIT, to run just-in-time compilation of CIL into native code

The standard name of the java compiler is javac. This compiler can translate any code written in the Java programming language into lower level machine language. The output of running the javac command is compiled code that can run on computers and other compatible devices.

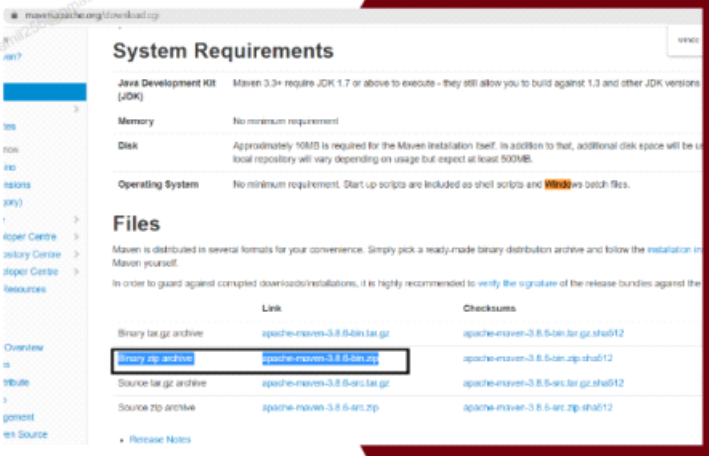
Python code written in high-level human-readable form into low-level machine code or bytecode that can be executed directly by





https://maven.apache.org/download.cgi





Job 1- Pulling Java project code from git via Jenkins

Step 1 Create new freestyle job in Jenkins via creation page

Step2 Go to configure page of job and Provide the java project link in git session as

https://github.com/DevSecOpsG/helloworldsimplejavaprogram

Step 3 save the job

Step 4 build the job via build now option if failed check the reason from console and update the Branch name

Step 5 -- check console result

Result will be

Commit message: "Create helloworld.java"
First time build. Skipping changelog.
Finished: SUCCESS





Job 2 – Compile and Building Java program / project from git

Step 1 Create new freestyle job in Jenkins

Step 2 Provide the java project and git location as

https://github.com/DevSecOpsG/helloworldsimplejavaprogram

Step 3 Create build steps as build

Build section and click on Add build step →

Execute Windows batch command	Execute shell
Windows	Linux
set path=C:\Program Files\Java\jdk1.8.0_311\bin javac helloworld.java java Simple	javac helloworld.java java Simple

Step 4
save and build

If fails go to next slide and do the configuration





Adding some extra configuration to run java program from Jenkins (if windows / locally)



go to Global tool config page add JDK and choose install automatically

run the job it will get failed and get the oracle account add oracle account..



Provide the Oracle account details and Run again

Job 3- Simple Java program compile java program locally if you have installed Jenkins locally

Step 1
Create new freestyle job

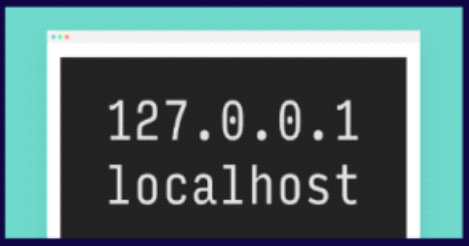
Step 2
Provide the java project (SCM) as none

Step 3
Create build steps as build

Build section and click on Add build step → Execute Windows batch command

```
cd C:\Users\admin\Desktop\notepad++\
set path=C:\Program Files\Java\jdk1.8.0_231\bin
javac helloworld.java
java Simple
```

Step 4
save and build



Job 4- Create Jenkins pipeline CI-example

Step 1 Create new freestyle job in Jenkins

Step 2 Provide the java project/program and git location as

https://github.com/DevSecOpsG/helloworldsimplejavaprogram



Step 3 Create build steps as build
Build section and click on Add build step →

Windows batch command	Execute shell
Windows	Linux
set path=C:\Program Files\Java\jdk1.8.0_231\bin javac helloworld.java java Simple	javac helloworld.java java Simple

Step 4 Create a Continuous Integration of Compile and Build via Poll SCM

i.e Poll Source code management (**SCM**)



Step 5 Save and change the code and wait as per schedule time

Schedule a build Periodically

Jenkins uses a cron expressions to schedule a job. Each line consists of 5 fields separated by TAB or whitespace:
Syntax: (Minute Hour DOM Month DOW)
MINUTE: Minutes in one hour (0-59)
HOURS: Hours in one day (0-23)
DAYMONTH: Day in a month (1-31)
MONTH: Month in a year (1-12)
DAYWEEK: Day of the week (0-7) where 0 and 7 are Sunday
Example: H/2 * * * * (schedule your build for every 2 minutes)

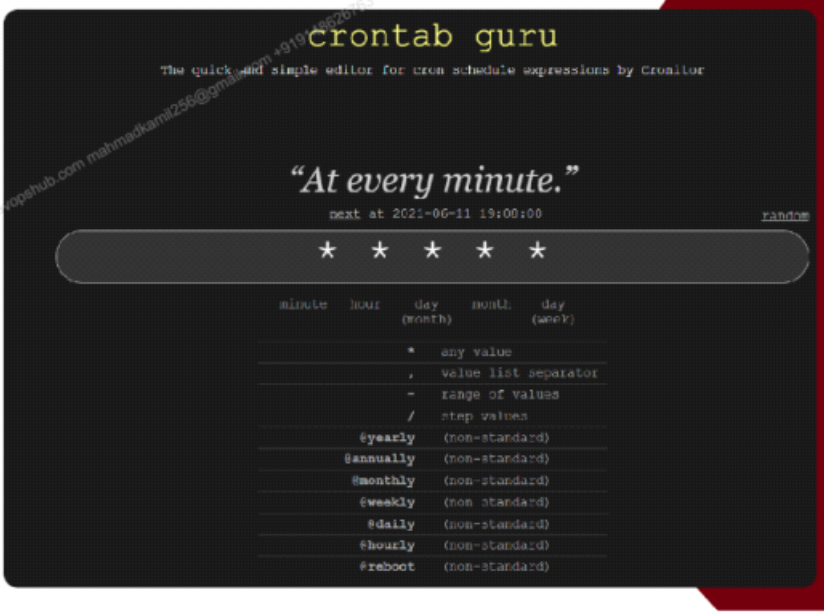


Jenkins schedule format

Jenkins schedule format is nothing but a cron schedule expression. It contains 5 fields

```
┌──────────── minute (0 - 59)
│ ┌────────── hour (0 - 23)
│ │ ┌──────── day of month (1 - 31)
│ │ │ ┌────── month (1 - 12)
│ │ │ │ ┌──── day of week (0 - 6) (Sunday to Saturday;
│ │ │ │ │                          7 is also Sunday on some systems)
│ │ │ │ │
│ │ │ │ │
* * * * *  schedule command to execute
```

https://crontab.guru/#*_*_*_*_*



Configure the Email Notification in Jenkins

Go to the Manage Jenkins > Configure System then find Extended E-mail Notification and fill in the following details:

Note: Below details are for using a Gmail account for emails hence using Gmail SMTP server and port. If you are using a different email provider then please check the details form here.

SMTP server – smtp.gmail.com
SMTP Port – 465
Click on advanced below the SMTP Port box.
SMTP Username – any Gmail id. This Gmail account will be used to send email notifications.
SMTP Password – password of the Gmail id used above
Check the Use SSL box.
Default Recipient – Add a default email id.





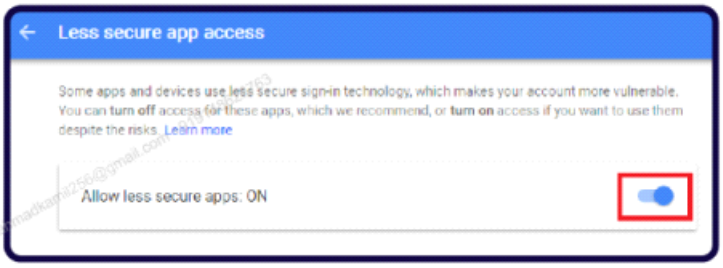
- **Gmail SMTP port** (TLS): 587.
- **Gmail SMTP port** (SSL): 465.

Step 2. Allow less secure app for the Gmail used in the above step.
The E-mail Notification will work if we enable Less Secure App Access for the Gmail account.

To enable Less Secure App Access. Go to this page here.

Then go to Security tab and Turn on Less secure app access.

gmail-allow-less-secure-apps



Step 3. Test the configuration.
You can test the configuration of by sending a demo email to desired email id. Just go to the E-mail Notification fill the same details in this section as above and at the bottom of the configuration section, there is an option to test the email notification.

Check the box and enter any email id on which you want to send an email.

JAVA

Java is a high-level, class-based, object-oriented programming language. It is a general-purpose programming language meaning that compiled Java code can run on all platforms that support Java

SPRINGBOOT

Spring Boot helps developers create applications that just run

MAVEN

Maven is a build automation tool used primarily for Java projects. Maven can also be used to build and manage projects written in C#, Ruby, Scala, and other languages.
mnv clean , maven install , mvn package

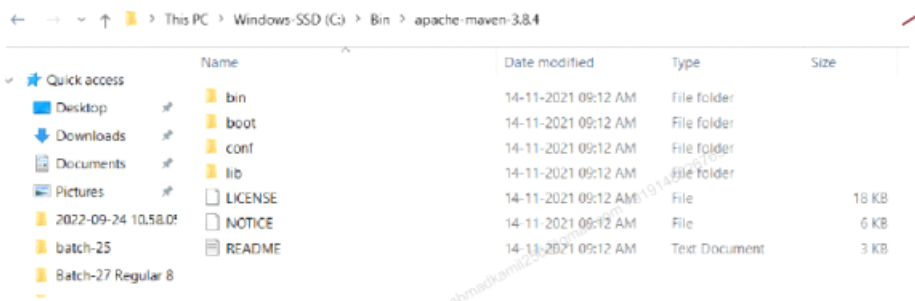
Maven Configuration go to jenkins > Manage Jenkins > Global tools Configure

http://localhost:8080/configureTools/

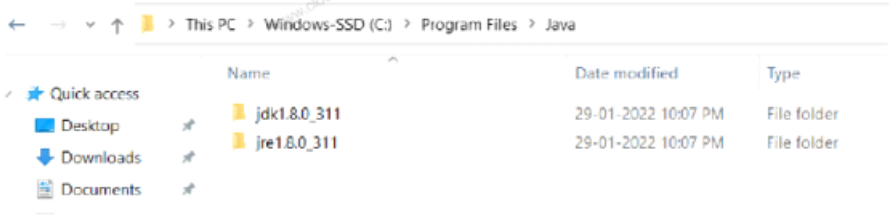


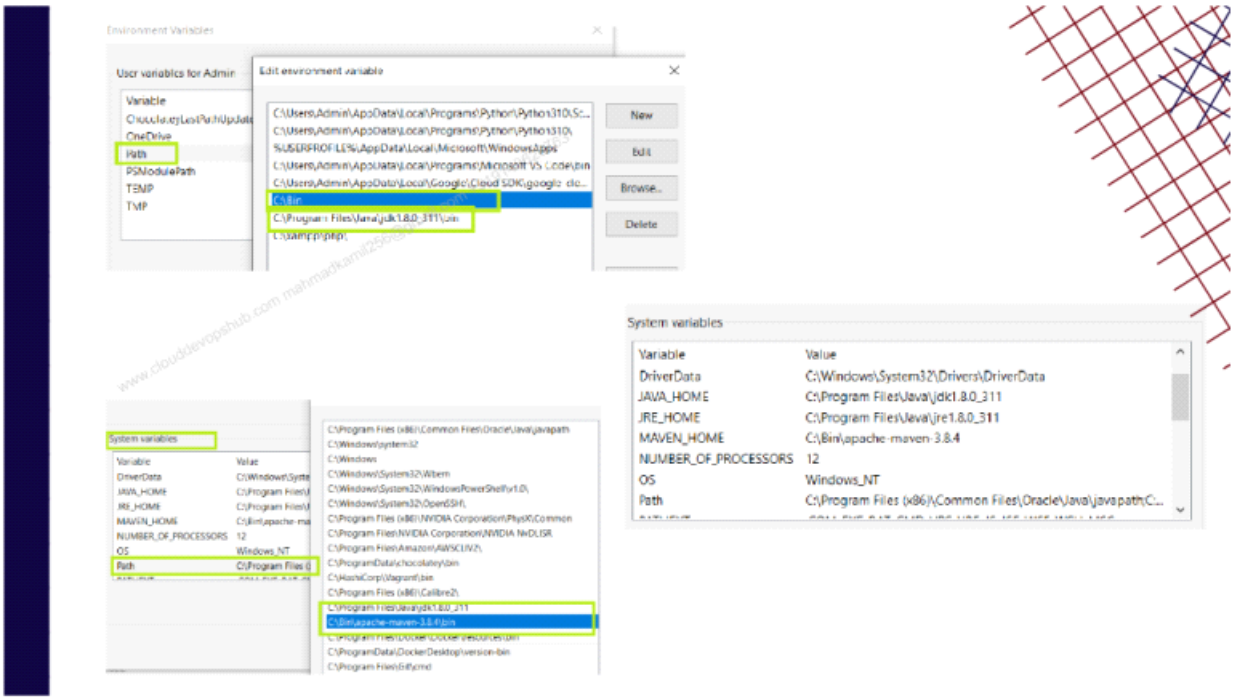
https://mkyong.com/maven/how-to-install-maven-in-windows/

Maven Configuration snapshot



JAVA installed snapshot





Project 1 Java project Spring boot application with maven integration With CI configuration

Step 1 :- go to https://github.com/spring-projects/spring-petclinic (put this if u have java 17)
https://github.com/DevSecOpsG/Pet-application-ci-maven.git (put this for java 8 & 11)

Clone the code and open in visual studio.. or if linux apt install maven

src :- java code and test
pom.xml file to create a build via maven

Step 2 :- Open jenkins create new job and provide the GIT url
Step 3 :- Open the Execute windows batch command

Execute Windows batch command (Try this)	Execute Shell linux
cd spring-petclinic	cd spring-petclinic
set path=C:\Bin\apache-maven-3.8.4\bin	mvn package
mvn package	
java -jar target/*.jar --server.port=8083 java -jar spring-petclinic-2.3.1.BUILD-SNAPSHOT.jar --server.port=8088	

Step 4 check logs

```
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  17:16 min
[INFO] Finished at: 2021-11-08T21:16:55+05:30
[INFO] ------------------------------------------------------------------------
Finished: SUCCESS
```

```
[INFO]
[INFO] Results:
[INFO]
[WARNING] Tests run: 40, Failures: 0, Errors: 0, Skipped: 1
```

Step 5 :- Now our target is to get the jar file

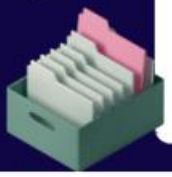
go to job workflow

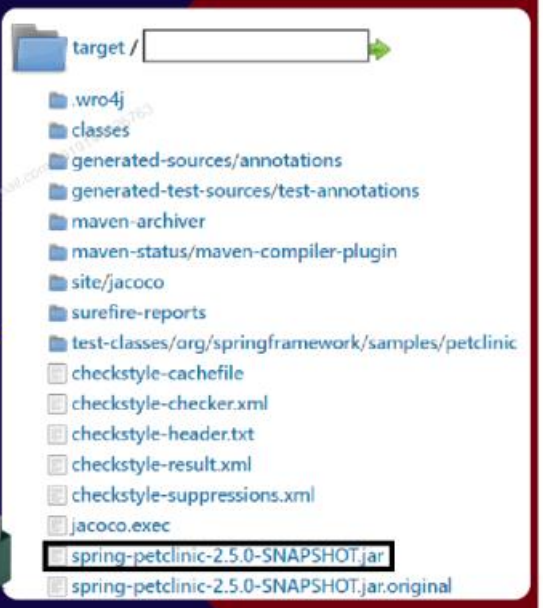
ws/target/ and download the jar file and Run

java -jar target/*.jar --server.port=8083

Step 6 :- go to browser and open http://localhost:8083/ your application will be up and running via jenkins

Step 7 :- setup CI via pol SCM option





[INFO] BUILD FAILURE
[INFO] --
[INFO] Total time: 4.276 s
[INFO] Finished at: 2023-06-05T18:57:56Z
[INFO] --
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-enforcer-plugin:3.3.0:enforce (enforce-java) on project spring-petclinic:
[ERROR] Rule 0: org.apache.maven.enforcer.rules.version.RequireJavaVersion failed with message:
[ERROR] This build requires at least Java 17, update your JVM, and run the build again

Simply Update your JAVA....

apt install openjdk-17-jdk openjdk-17-jre

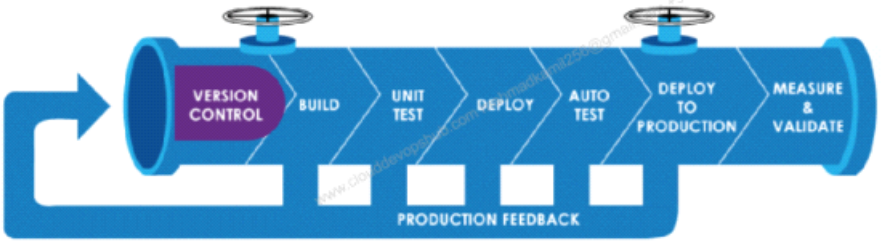
Verify
java -version

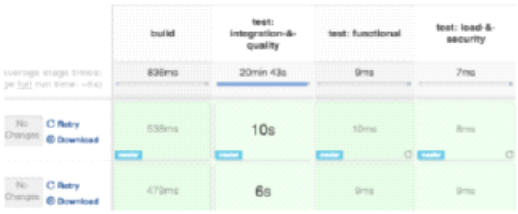
openjdk version "17.0.7" 2023-04-18
OpenJDK Runtime Environment (build 17.0.7+7-Ubuntu-0ubuntu118.04)
OpenJDK 64-Bit Server VM (build 17.0.7+7-Ubuntu-0ubuntu118.04, mixed mode, sharing)



What is Jenkins Pipeline?

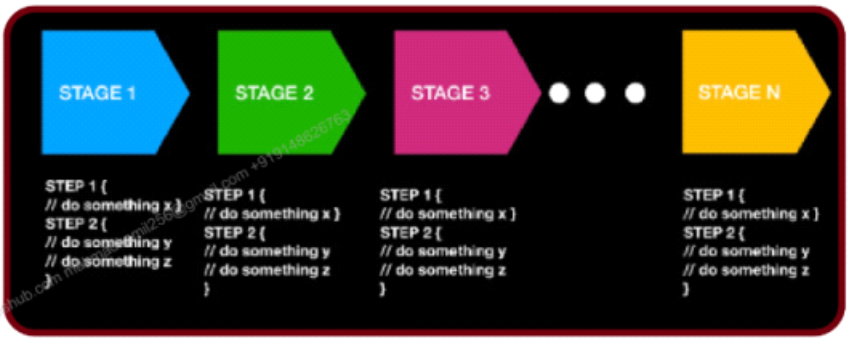
Jenkins Pipeline (or simply "Pipeline") is a suite of plugins which supports implementing and integrating continuous delivery pipelines into Jenkins via Groovy Code.







A **pipeline Job** is a sequence of events or jobs that can be executed from a single job.



Here, you should see two familiar concepts: *Stage* and *Step*.

- **Stage:** A block that contains a series of steps. A stage block can be named anything; it is used to visualize the pipeline process.
- **Step:** A task that says what to do. Steps are defined inside a stage

Syntax to Write any Jenkins Pipeline

It's a Groovy language

Writing the Pipeline:
Under Advanced Project Options, choose a sample pipeline script or write a sample syntax like this:

```
pipeline {
    agent any
    stages{
      stage(''){
        steps{

        }
      }
    }
}
```

What is Groovy?

It's like a super version of Java:

- *Supports most Java syntax but allows simpler syntax for many constructs*
- *Supports all Java libraries but provides many extensions and its own productivity libraries*
- *Has both a static and dynamic nature*
- *Great extensibility for language and tooling*



Step 1. Create a Pipeline project.
Click on New Item.
Give a name to the Job. For eg. demo-pipeline.
Select Pipeline from the different types of jobs.

Step 2. Configuring the Pipeline Job
All default configuration
Check the Poll SCM and schedule to check for any changes for every minute using the following cron expression: * * * * *
Writing the Pipeline:
Under Advanced Project Options, choose a sample pipeline script or write a sample syntax like this:

Linux SH

```
pipeline {
agent any

   stages {
      stage('Git clone') {
         steps {
         git branch: 'main', url:
'https://github.com/DevSecOpsG/helloworldsimplejavaprogram'
         }
      }
      stage('Build'){
         steps{
         sh 'javac helloworld.java'
         }
      }
      stage('Run'){
         steps{
         sh 'java Simple'
         }
      }
   }
}
```



Windows BAT

```
pipeline {
agent any

   stages {
      stage('Git clone') {
         steps {
         git branch: 'main', url:
'https://github.com/DevSecOpsG/helloworldsimplejavaprogram'
         }
      }
      stage('Build'){
         steps{
         bat '"set path=C:\\Program
Files\\Java\\jdk1.8.0_311\\bin\\
         javac helloworld.java"'
         }
      }
      stage('Run'){
         steps{
         bat 'java Simple'
         }
      }
   }
}
```



URL Options

The following commands when appended to the Jenkins instance URL will carry out the relevant actions on the Jenkins instance.

below restart command will only work if you will use a approach 1
For approach 2 close the cmd prompt and start Jenkins again

http://localhost:8080/jenkins/exit – shutdown jenkins
http://localhost:8080/jenkins/restart – restart jenkins
http://localhost:8080/jenkins/reload – to reload the configuration
http://localhost:8080/prepareShutdown/

http://localhost:8080/restart — working
safeRestart restart prepareShutdown



Backup Jenkins Home
The Jenkins Home directory is nothing but the location on your drive where Jenkins stores all information for the jobs, builds etc. The location of your home directory can be seen when you click on Manage Jenkins → Configure system.

Post build Action

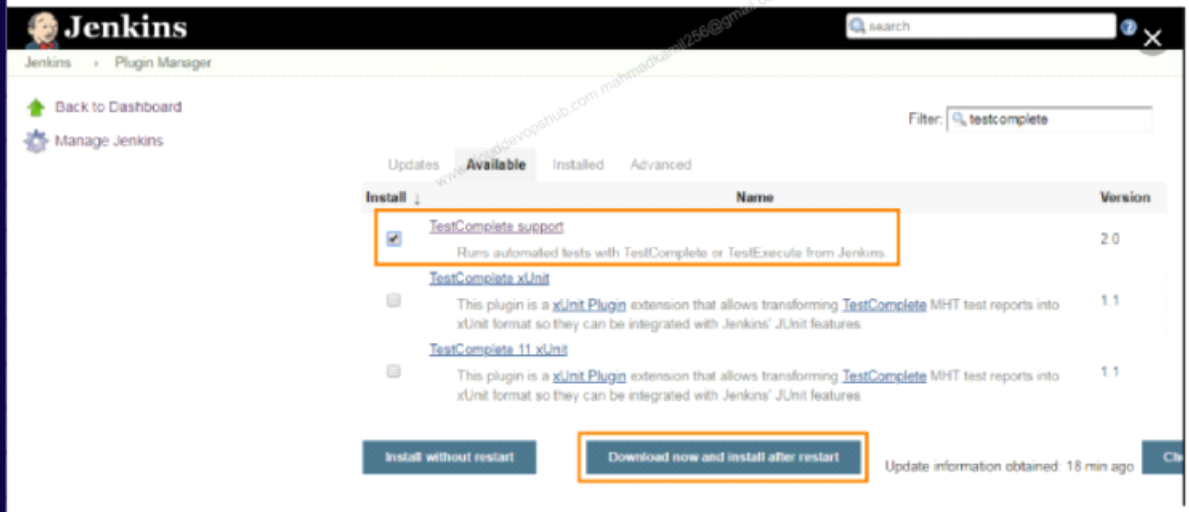
After build next part is to run another job for another env or perform another activity

via Post build Action

Provide the Project to build according to condition



Jenkins plugin page

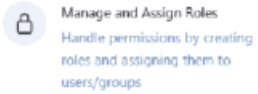


Role-based Authorization Strategy

https://plugins.jenkins.io/role-strategy/

About this plugin
The Role Strategy plugin is meant to be used from Jenkins to add a new role-based mechanism to manage users' permissions. Supported features

- Creating global roles, such as admin, job creator, anonymous, etc.,
- Allowing to set Overall, Agent, Job, Run, View and SCM permissions on a global basis.
- Creating item roles, allowing to set item specific permissions (e.g Job, Run or Credentials) on Jobs, Pipelines and Folders.
- Creating agent roles, allowing to set agent specific permissions.
- Assigning these roles to users and user groups



Jenkins has **1800+ Plugin's for integration's**
How many Regular usable plugin's you know ?



Jenkins useful plugins https://plugins.jenkins.io/

1,800+ **plugins are available** (community contributed Jenkins plugins)

- Git Plugin
- GitHub Integration Plugin
- Build Flow.
- Build Monitor.
- Build Name Setter
- Build Pipeline
- Delivery Pipeline
- Email Extension
- Parameterized Trigger
- SSH
- xUnit
- Job DSL plugin
- Monitoring Plugin
- Kubernetes Plugin



Brief discussion about Jenkins plugins

Git Plugin
Provides access to GitHub as an SCM which acts as a repository browser for many other providers

GitHub Integration Plugin
This is the most fundamental plugin needed for integrating Jenkins with GitHub projects. With this plugin, you can schedule your build, pull code and data files from GitHub repositories to Jenkins, and automatically trigger each build as needed.

Build Flow
Managing a pipeline in Jenkins requires moderate configurations across all the jobs participating in the flow. Configurations can combine different types, including parameterized build, parallel builds, joins or downstream wait.



Build Monitor
Visibility is the core aspect of the Jenkins CI tool. It being understood that CI is a part of collective ownership, everyone should know that if something breaks, some part of it is not working consistently.

Build Name Setter
very build comes with a build number, but most of the time its hard to identify a particular build with a build number. Many types of builds get executed with an update from revision number by source code, so it would be great deal easier to use revision number as a unique identifier.



Brief discussion about Jenkins plugins



Build Pipeline
As stated earlier, visibility is the core aspect of Jenkins pipeline to be successful in terms of its value.This plugin helps renders upstream and downstream connected jobs that typically form a build pipeline in a very nice fashion on the monitor, big monitors especially

Delivery Pipeline
This plugin visualizes Delivery Pipelines (Jobs with upstream/downstream dependencies). This is somewhat similar to Build pipeline plugin but has avery lean design

Email Extension
This plugin is a replacement for Jenkins's email publisher. This is more advanced plugin with better features for sending build status messages

Parameterized Trigger
his plugin lets you trigger new builds when your build has completed, with various ways of specifying parameters for the new build.

SSH
You can use the SSH Plugin to run shell commands on a remote machine via ssh.



xUnit
This plugin makes it possible to record xUnit test reports.



Brief discussion about Jenkins plugins

GitHub Integration Plugin
This is the most fundamental plugin needed for integrating Jenkins with GitHub projects.With this plugin, you can schedule your build, pull code and data files from GitHub repositories to Jenkins, and automatically trigger each build as needed.

Job DSL plugin
This plugin allows users to describe tasks, handle scripts, and update Jenkins jobs. With Job DSL you can define jobs by bare minimum programmatic form with the help from templates. You also gain various jobs views such as 'BuildMonitorView', 'BuildPipelineView', 'CategorizedJobsView', and many others. These views provide high visibility of the status of selected Jenkins jobs.

Monitoring Plugin
The 'Monitoring Plugin' uses JavaMelody to keep track of Jenkins. This plugin allocates charts for CPU, HTTP response time, memory, and much more



Pipeline Plugin
Pipeline' attempts to automate your continuous delivery pipeline and performs other complex tasks in Jenkins using traditional plugins and freestyle projects

Kubernetes Plugin
The 'Kubernetes plugin' works, surprisingly, with Kubernetes. So, if you plan to use Kubernetes for your infrastructure, this plugin is the way forward for setting up and tearing down Jenkins' agents. While moving to Kubernetes is not the easiest process, the results are worth it.

THANK YOU